

Informe técnico para Santiago: Fine-tuning de wav2vec 2.0 con adaptadores y PEFT para clasificación binaria de hablantes con enfermedad neurodegenerativa vs. control

TL;DR

- **Sí es viable y está bien fundamentado** hacer fine-tuning parameter-efficient (PEFT) de wav2vec 2.0 para clasificación binaria (p. ej. AD vs. CTR) en régimen de pocos datos. La clase correcta en HuggingFace es `Wav2Vec2ForSequenceClassification` (cabeza lineal sobre pooling), **no** `Wav2Vec2ForCTC` —que es para ASR—; varios de los tutoriales que tienes (heyamit10, khanld, TensorFlow Hub) enseñan CTC/ASR y no aplican directamente a tu tarea de clasificación.
 - **Para los tres métodos que mapeaste:** (1) **adapters bottleneck** (D→m→D + residual + init casi-cero, Houlsby et al. 2019) se aplican con la librería `adapters`; (2) **LoRA** (Hu et al. 2021) se aplica con `peft` vía `LoraConfig`+`get_peft_model` sobre `q_proj`/`v_proj`, con reducción de parámetros entrenables de órdenes de magnitud; (3) **prefix-tuning** (Li & Liang 2021) antepone vectores continuos entrenables en cada capa. La evidencia empírica en habla (PEFT-SER, ELP-Adapters) muestra que con **2-3 % de los parámetros** se iguala o supera el full fine-tuning en clasificación.
 - **Estado del arte en neurodegenerativas:** existe abundante trabajo con wav2vec 2.0/HuBERT/WavLM en Alzheimer (ADReSS/ADReSSo/Pitt), Parkinson (PC-GITA) y deterioro cognitivo leve (MCI), casi siempre con **embeddings congelados o full fine-tuning**. El uso de **PEFT específicamente para clasificación neurodegenerativa es una laguna casi inexplorada** (solo dos preprints de 2026, HASS y VoxCog) — lo que convierte tu tesis en una contribución oportuna y original.
-

Key Findings

1. **Clasificación ≠ ASR.** Tu tarea es *secuencia → etiqueta única* (binaria). Usa `Wav2Vec2ForSequenceClassification`: aplica una proyección, hace **mean pooling** sobre el eje temporal y pasa por una capa densa + capa de clasificación. Esto difiere del pipeline CTC de tus tutoriales de referencia.
2. **El feature encoder CNN debe congelarse** (`model.freeze_feature_encoder()`). Es la práctica estándar y recomendada por la documentación de Transformers y la literatura (Vaessen & van Leeuwen 2022 mostraron que congelarlo da resultados más estables entre semillas). (`arXiv`)
3. **wav2vec2-base:** CNN de 7 capas (16 kHz → ~50 Hz, un vector cada 20 ms), **12 capas Transformer**, dimensión oculta **768**, 12 cabezas, FFN intermedia 3072, **~95M parámetros**. La variante *large* tiene 24 capas y dim. 1024 (~315M). (`Substack`) (`arxiv`)
4. **Las capas tempranas importan en habla clínica.** Múltiples estudios (Klempf et al. 2025 en PD; análisis layer-wise en dysarthria) encuentran que las **primeras capas**

Transformer o incluso el feature extractor capturan la información patológica más discriminativa — relevante para decidir qué capa usar como representación.

5. **PEFT iguala o supera al full fine-tuning en clasificación de habla con una fracción de parámetros.** En reconocimiento de emociones, la combinación BA+LoRA+WS supera al FT usando 2-3 % de los parámetros entrenables (arXiv 2402.11747).
6. **Riesgo metodológico crítico — efecto "Clever Hans" y fuga de datos.** El fine-tuning de wav2vec 2.0 para AD puede alcanzar 97.2 % en el Pitt corpus pero desplomarse a 77.7 % en ADReSSo, evidenciando sobreajuste a artefactos del corpus y no a biomarcadores reales.

Details

PARTE 1 — Implementación y estado del arte del fine-tuning de wav2vec 2.0 con PEFT

1.1 La distinción fundamental: clasificación vs. transcripción

Conviene aclararlo de entrada porque condiciona todo el código. Los recursos que aportaste se dividen así:

- **heyamit10 (Medium), khanld/ASR-Wav2vec-Finetune, TensorFlow Hub** → enseñan ASR con CTC loss ((`Wav2Vec2ForCTC`), (`Wav2Vec2CTCTokenizer`)). El objetivo es alinear audio ↔ texto. **No es tu caso**, aunque son útiles para entender el preprocesamiento de audio (carga con (`librosa`), resampleo a 16 kHz, conversión a mono).
- **Tu tarea (AD vs. CTR)** → clasificación de secuencia. La clase es (`Wav2Vec2ForSequenceClassification`), descrita en la documentación de Transformers como "Wav2Vec2 Model with a sequence classification head on top (a linear layer over the pooled output) for tasks like SUPERB Keyword Spotting". (`Hugging Face`)

El notebook oficial de HuggingFace (`examples/audio_classification.ipynb`) (sobre Keyword Spotting de SUPERB) es el ejemplo canónico para tu pipeline: "any speech model checkpoint ... as long as that model has a version with a Sequence Classification head (e.g. `Wav2Vec2ForSequenceClassification`)". (`GitHub`)



1.2 Arquitectura de wav2vec 2.0 base (lo que debes saber para insertar PEFT)

Componente	Detalle	¿Se entrena?
Feature encoder (CNN)	7 capas conv, 16 kHz → ~50 Hz (1 vector/20 ms), salida 512-dim	Congelado (estándar)

Componente	Detalle	¿Se entrena?
Proyección	512 → 768 (primer "hidden state", índice 0)	Según estrategia
Context network	12 capas Transformer , dim 768 , 12 cabezas, FFN 3072	Aquí van los adapters / LoRA / prefijos
Pooling	Mean pooling sobre el eje temporal	—
Cabeza de clasificación	Densa (p.ej. 256) + capa lineal a <code>num_labels</code>	Siempre se entrena

Cada capa Transformer contiene proyecciones de atención nombradas `q_proj`, `k_proj`, `v_proj`, `out_proj`, más el bloque feed-forward (`intermediate_dense`, `output_dense`). Estos son los nombres de módulo que necesitas para apuntar LoRA o adapters.

1.3 Pipeline base (full / partial fine-tuning) con `Wav2Vec2ForSequenceClassification`

```
python

import torch
from transformers import (Wav2Vec2ForSequenceClassification,
                          AutoFeatureExtractor, TrainingArguments, Trainer)

model_name = "facebook/wav2vec2-base"
feature_extractor = AutoFeatureExtractor.from_pretrained(model_name)

model = Wav2Vec2ForSequenceClassification.from_pretrained(
    model_name,
    num_labels=2,                # AD vs CTR
    label2id={"CTR": 0, "AD": 1},
    id2label={0: "CTR", 1: "AD"},
)

# Práctica estándar: congelar el feature encoder CNN
model.freeze_feature_encoder()

# Preprocesamiento (audio ya cargado como np.array mono a 16 kHz)
def preprocess(batch):
    inputs = feature_extractor(
        batch["audio"], sampling_rate=16000,
        max_length=16000*10, truncation=True, padding="max_length")
    inputs["labels"] = batch["label"]
    return inputs
```

Fine-tuning parcial (recomendado en bajos recursos): en vez de entrenar las 12 capas, descongela solo las **últimas 3 capas Transformer**. Sampath et al. (arXiv 2503.03756, *Efficient*

Finetuning for Dimensional Speech Emotion Recognition, que tienes en tu bibliografía) demuestran que "*partial finetuning ... achieves comparable performance to full finetuning, showing no statistically significant differences*", recomendando explícitamente entrenar **las últimas tres capas Transformer en precisión mixta** (67 % de speedup) y añadir caching de representaciones intermedias (88 % de speedup, -71 % de parámetros entrenables). (arxiv)

(arxiv)

python

```
# Fine-tuning parcial: congelar todo salvo las últimas 3 capas + cabeza
for p in model.wav2vec2.parameters():
    p.requires_grad = False
for layer in model.wav2vec2.encoder.layers[-3:]:
    for p in layer.parameters():
        p.requires_grad = True
# La cabeza (model.classifier / model.projector) queda entrenable por defecto
```

1.4 Método 1 — Bottleneck adapters (Houlsby et al. 2019)

Mecanismo. Un adapter es un módulo bottleneck insertado dentro de cada capa Transformer. Dada una representación oculta $h \in \mathbb{R}^D$:

$$h \leftarrow W_{up} f(W_{down} h) + r$$

donde $W_{down} \in \mathbb{R}^{m \times D}$ proyecta a una dimensión reducida $m \ll D$ (el "cuello de botella"), f es una no-linealidad (ReLU/GeLU), $W_{up} \in \mathbb{R}^{D \times m}$ re-proyecta a D , y r es la **conexión residual**. La **inicialización casi a cero** de W_{up} hace que al inicio el adapter sea aproximadamente la identidad, preservando el modelo preentrenado — la documentación de AdapterHub confirma que "*the inserted adapter module should be equivalent to the identity function at the start of the finetuning*", condición clave para buen desempeño. (arxiv) (arxiv)

Dónde se inserta.

- **HoulsbyConfig** (`DoubleSeqBnConfig`): **dos** adapters por capa — tras la multi-head attention y tras el bloque feed-forward. Más expresivo, $\sim 2\times$ parámetros. (Adapterhub) (Mbrenndoerfer)
- **PfeifferConfig** (`SeqBnConfig`): **un** adapter por capa, solo tras el FFN. Más eficiente; es el default en la mayoría de frameworks y el usado en los benchmarks de audio (Audio Spectrogram Transformer, arXiv 2312.03694). (Adapterhub + 2)

Presupuesto de parámetros. Controlado por el `reduction_factor` = D/m . Con $D=768$ y `reduction_factor=16`, $m=48$; cada adapter añade $\approx 2 \cdot D \cdot m \approx 74K$ parámetros, $\sim 0.9M$ en 12 capas (Pfeiffer) — orden del $\sim 1\%$ del modelo. Houlsby duplica esa cifra. En la adaptación de Whisper-small a habla infantil (arXiv 2406.10507) el adapter tuning logró desempeño similar al full FT "*while having only 1.29M parameters for updates*". (arxiv)

```
python

# pip install adapters
import adapters
from adapters import BnConfig
from transformers import Wav2Vec2ForSequenceClassification

model = Wav2Vec2ForSequenceClassification.from_pretrained(
    "facebook/wav2vec2-base", num_labels=2)
adapters.init(model)                # habilita la API de adapters

# Bottleneck Houlsby: adapter tras atención Y tras FFN, init tipo houlsby
config = BnConfig(
    mh_adapter=True,                # adapter tras multi-head attention
    output_adapter=True,            # adapter tras FFN → configuración Houlsby
    reduction_factor=16,            # D/m (768/16 = 48 dim de cuello de botella)
    non_linearity="relu",
)
model.add_adapter("ad_vs_ctr", config=config)
model.train_adapter("ad_vs_ctr")    # congela el resto, entrena solo el adapter +
```

Para Pfeiffer, usa `(mh_adapter=False, output_adapter=True)`. La librería `(adapters)` soporta wav2vec 2.0 oficialmente.

1.5 Método 2 — LoRA, Low-Rank Adaptation (Hu et al. 2021)

Mecanismo. En lugar de actualizar una matriz de pesos $W_0 \in \mathbb{R}^{d \times k}$, LoRA la congela y aprende una actualización de **rango bajo**:

$$W = W_0 + \Delta W = W_0 + BA, \quad B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times k}, r \ll \min(d, k)$$

El forward pass es $h = W_0x + BAx$, escalado por $\frac{\alpha}{r}$. Hipótesis fundacional: la actualización de pesos durante la adaptación tiene un "**rango intrínseco**" bajo. A se inicializa con ruido gaussiano y B a cero, de modo que $\Delta W = 0$ al inicio. Ventaja única frente a adapters/prefijos: en inferencia, BA puede **fusionarse** en W_0 sin overhead de latencia (cita oficial: "*there is no added overhead during inference, setting it apart from previous adapters*"). (arxiv + 3)

Dónde se inserta. Lo estándar (y el default de PEFT para muchas arquitecturas) es aplicarlo a las proyecciones de atención `(q_proj)` y `(v_proj)`. En habla, el trabajo de detección de audio falso con wav2vec2/XLSR (Wang et al. 2023) encontró que **rango $r=4$** y aplicar LoRA a W_q, W_v (o W_q, W_k, W_v) era óptimo, reduciendo los parámetros entrenables $\sim 198\times$ (−99.49 %) con caída de desempeño marginal frente al full FT.

Presupuesto de parámetros. Por cada capa adaptada: $r \cdot (d + k)$ parámetros. Con $d=k=768$ y $r=16$: $16 \cdot (768+768) \approx 24.6K$ por proyección; aplicando a q_proj+v_proj en 12 capas $\approx 590K$, $<1\%$ del modelo. Reducir a $r=8$ o $r=4$ lo reduce a la mitad/cuarto.

Código (librería `peft`):

```
python

# pip install peft
from peft import LoraConfig, get_peft_model
from transformers import Wav2Vec2ForSequenceClassification

model = Wav2Vec2ForSequenceClassification.from_pretrained(
    "facebook/wav2vec2-base", num_labels=2)
model.freeze_feature_encoder()

lora_config = LoraConfig(
    r=16,                                # rango (prueba 4, 8, 16)
    lora_alpha=32,                        # escalado alpha/r
    target_modules=["q_proj", "v_proj"],  # proyecciones de atención
    lora_dropout=0.05,
    bias="none",
    modules_to_save=["classifier", "projector"], # la cabeza se entrena completa
)
model = get_peft_model(model, lora_config)
model.print_trainable_parameters()

# -> trainable params: ~0.6M || all params: ~95M || trainable%: ~0.6%
```

Nota práctica: como wav2vec2 no es un `CAUSAL_LM`, **no** fijas `task_type="CAUSAL_LM"`; déjalo sin especificar y usa `modules_to_save` para que la cabeza de clasificación (inicializada aleatoriamente) se entrene por completo. Este patrón (LoRA sobre `q_proj/v_proj` de un modelo wav2vec2 de HuggingFace) está documentado en el propio repo de `peft` (issue #1569). ([GitHub](#))

1.6 Método 3 — Prefix-tuning (Li & Liang 2021)

Mecanismo. Antepone una secuencia de **vectores continuos entrenables** ("**prefijos**" o **soft prompts**) a los estados ocultos de **cada** capa Transformer, manteniendo congelados todos los pesos del modelo. A diferencia del *prompt-tuning* (Lester et al. 2021), que solo añade prompts en la capa de entrada, prefix-tuning optimiza pares clave-valor de prefijo en **todas** las capas. Para estabilizar el entrenamiento, los prefijos no se optimizan directamente sino que se reparametrizan vía un MLP: $P_\theta[i, :] = \text{MLP}_\theta(P'_\theta[i, :])$ (descartando el MLP tras el entrenamiento). Es especialmente eficiente en almacenamiento: solo guardas los vectores de prefijo. ([arxiv + 2](#))

Dónde se inserta. En las claves y valores de la atención de cada capa (se concatenan L_{prefix} vectores antes de las claves/valores reales). En audio (NOCASA 2025, arXiv 2509.03256) se ha

usado prefix-tuning sobre wav2vec2 para inyectar información de palabra, con tan solo $L_{prefix} = 2$ vectores de prefijo. (arxiv)

Presupuesto de parámetros. $L_{prefix} \times L_{capas} \times 2 \times D$ (factor 2 por clave y valor). Con $L_{prefix} = 16, 12$ capas, $D=768$: $\approx 295K$ parámetros ($\sim 0.3\%$), más el MLP de reparametrización (temporal).

Código (librería (peft)):

```
python

from peft import PrefixTuningConfig, get_peft_model

prefix_config = PrefixTuningConfig(
    num_virtual_tokens=16,      # L_prefix: nº de vectores de prefijo por capa
    encoder_hidden_size=768,
)
model = get_peft_model(model, prefix_config)
model.print_trainable_parameters()
```

Advertencia honesta: el soporte de prefix-tuning en (peft) está orientado a modelos de lenguaje; para wav2vec2 puede requerir ajustes o usar (adapters) (que implementa Prefix Tuning de Li & Liang explícitamente). Si encuentras fricción, prioriza LoRA y bottleneck adapters, que tienen soporte de primera clase para wav2vec2. De hecho, mi búsqueda no halló **ningún** trabajo que use prefix-tuning para clasificación neurodegenerativa (solo aparece en ASR/benchmarks), así que es la opción más experimental de las tres. (Adapterhub)

1.7 Tabla comparativa de los tres métodos PEFT

Método	Ecuación / mecanismo	Inserción en wav2vec2	% parám. típico	Soporte / librería	Fusionable en inferencia
Bottleneck adapter	$h \leftarrow W_{up}f(W_{down}h) + r$	Tras atención y/o FFN	$\sim 1-2\%$	(adapters) (Houlsby/Pfeiffer)	No
LoRA	$W_0 + \frac{\alpha}{r}BA$	(q_proj), (v_proj)	$<1\%$	(peft) ((LoraConfig))	Sí
Prefix-tuning	prefijos K/V por capa	claves/valores de atención	$\sim 0.3\%$	(peft) / (adapters)	No

1.8 Bucle de entrenamiento común (Trainer)

```
python
```

```
import numpy as np, evaluate
from transformers import TrainingArguments, Trainer

acc = evaluate.load("accuracy"); f1 = evaluate.load("f1")
def compute_metrics(p):
    preds = np.argmax(p.predictions, axis=1)
    return {"accuracy": acc.compute(predictions=preds, references=p.label_ids)["ac
        "f1": f1.compute(predictions=preds, references=p.label_ids, average="m

args = TrainingArguments(
    output_dir="w2v2-ad-ctr", learning_rate=3e-4,    # PEFT tolera LR mayor que ful
    per_device_train_batch_size=4, gradient_accumulation_steps=4,
    num_train_epochs=15, warmup_ratio=0.1, eval_strategy="epoch",
    fp16=True, load_best_model_at_end=True, metric_for_best_model="f1")

trainer = Trainer(model=model, args=args, compute_metrics=compute_metrics,
                  train_dataset=train_ds, eval_dataset=eval_ds)
trainer.train()
```

Notas: para full FT usa $LR \approx 3e-5$; para LoRA/adapters puedes subir a $1e-4$ - $5e-4$. En datasets pequeños, **batch size 1 + gradient accumulation 4** y **validación cruzada 5-fold por hablante** es el protocolo usado en la literatura de AD (arXiv 2406.07410).

PARTE 2 — Estado del arte: modelos auto-supervisados de habla en enfermedades neurodegenerativas

2.1 Panorama general y datasets clave

Dataset	Enfermedad	Idioma	Contenido	Acceso
ADReSS (Interspeech 2020)	AD	Inglés	Cookie Theft, balanceado edad/género	TalkBank/DementiaBank
ADReSSo (Interspeech 2021)	AD	Inglés	Solo audio (sin transcripción manual)	DementiaBank
ADReSS-M (ICASSP 2023)	AD	Inglés (train) / Griego (test)	Multilingüe, 291 muestras	DementiaBank

Dataset	Enfermedad	Idioma	Contenido	Acceso
DementiaBank / Pitt corpus	AD/demencia	Inglés	Habla espontánea longitudinal	DementiaBank
PC-GITA	Parkinson	Español (Colombia)	50 PD + 50 HC balanceado, vocales/lectura/monólogo	Bajo solicitud (J.R. Orozco-Arroyave, U. de Antioquia)
PROCESS (ICASSP 2025)	MCI + demencia	Inglés	82 HC, 59 MCI, 16 demencia; 3 prompts	Challenge
TAUKADIAL (Interspeech 2024)	MCI/NC	Inglés + Mandarín	507 muestras (285 MCI, 222 NC)	Challenge
NCMMSC / ADD-CC	AD	Mandarín	—	Challenge

PC-GITA es especialmente relevante para ti por su origen colombiano (GITA Lab, Universidad de Antioquia, Medellín): 50 pacientes con PD (UPDRS total medio 37.7±18.3, H&Y 2.2±0.7) y 50 controles, balanceado por sexo y edad (61±9 años), grabado a 44.1 kHz con tareas de lectura, monólogo espontáneo y vocales sostenidas. [\(medrxiv\)](#)

2.2 Alzheimer y demencia (AD)

- Agbavor & Liang (2022), *PLOS Digital Health* 1(12):e0000168** — "Predicting dementia from spontaneous speech using large language models". Usaron wav2vec2-base-960h vía [\(Wav2Vec2ForCTC\)](#) para **transcribir** el audio, y luego embeddings de GPT-3 (Ada/Babbage) + SVM/LR/RF para clasificar. Es un esquema de *embedding como feature*, no fine-tuning end-to-end del modelo de habla. El propio artículo señala que *"these AI-driven speech-based LOAD studies have achieved outstanding accuracy (around 90% on average) in detecting AD"*. [\(PLOS + 2\)](#)
- Zhu, Obyat, Liang, Batsis & Roth (2021), Interspeech, pp. 3790–3794, doi:10.21437/interspeech.2021-332** — "WavBERT: Exploiting Semantic and Non-semantic Speech using Wav2vec and BERT for Dementia Detection". Convierten wav2vec→transcripción→BERT, con una red de conversión de embeddings que preserva información no-semántica (pausas). Reportan: *"the WavBERT models achieved the highest accuracy of 83.1% in the classification task"* (modelo Mp1) sobre ADReSSo, con RMSE 4.44 (regresión) y F1 medio de 70.91 % (progresión). [\(CogVox\)](#) [\(PubMed\)](#)
- Liu, Feng, Yuan & Ling (2024), arXiv 2406.07410** — "Clever Hans Effect Found in Automatic Detection of Alzheimer's Disease through Speech". **Fine-tuning end-to-end** de [\(wav2vec2-large-xlsr-53\)](#) (inglés) y [\(wav2vec2-large-chinese\)](#) (mandarín) con **cabeza de clasificación de secuencia (capa lineal + sigmoide sobre average pooling)**, 5-fold CV, batch 1, grad-accum 4, 15 épocas, LR 3e-5. Hallazgo crítico de fuga de datos: *"it is close to 100% (97.2%) [en Pitt/Pco] ... much higher than that of the two challenge datasets*

Ao (80.7%), Aoo (77.7%)", además de 81 % en el dataset mandarín. **Lección directa para tu tesis:** la altísima exactitud intra-corpus puede reflejar artefactos del corpus (Clever Hans), no biomarcadores reales — valida siempre cross-corpus y por hablante. (arxiv + 2)

- **Fine-tuning conjunto SSL con multitask + data augmentation** (Chen et al., Interspeech 2021 y derivados): wav2vec2 alcanza 73.7 % acc / 72.8 % F1 y HuBERT 74.2 % acc / 73.6 % F1 en ADReSS — por debajo de BERT sobre transcripciones (83.3 %), lo que ilustra que el canal acústico puro suele rendir algo menos que el lingüístico en AD. (arxiv)

2.3 Parkinson (PD)

- **Gallo-Aristizábal, Escobar-Grisales, Ríos-Urrego, Nöth & Orozco-Arroyave (2024), TSD 2024, LNCS 15049, doi:10.1007/978-3-031-70566-3_27** — "Automatic Classification of Parkinson's Disease Using Wav2vec Embeddings at Phoneme, Syllable, and Word Levels", del GITA Lab. Con wav2vec 2.0 sobre **PC-GITA** reportan "*accuracies of 86%, 80%, and 83% for phonemes, syllables, and words, respectively*". (Nota: cifras más altas de "92 % en vocales / 97 % en palabras" que circulan corresponden a otros métodos multimodales del mismo grupo, no a wav2vec puro — no las atribuyas a wav2vec.) (Springer)
- **Klempíř, Skryjová, Tichopad & Krupička (2025), medRxiv 2025.01.29.25321319** — "Ranking Pretrained Speech Embeddings in Parkinson's Disease Detection: Does Wav2Vec 2.0 Outperform its 1.0 Version?". Comparan w2v1 vs w2v2 (embeddings congelados + clasificadores clásicos, ranking TOPSIS) en 3 corpus multilingües incluido PC-GITA: "*Wav2Vec 2.0 excelled across speech modes, with its first transformer layer demonstrating the best performance for classifying read text and monologue, and its feature extractor performing best in vowel-based classification*". **Implicación de diseño:** la primera capa Transformer y el feature extractor son las representaciones más informativas para PD. (nih)
- **Pooling de embeddings (medRxiv 2025.09.13.25335694)** — "Statistical, multi-scale and attention-based layer pooling of Wav2Vec-2 ... for Parkinson's Disease Detection". Hallazgos: las capas tempranas de wav2vec2 dan los rasgos más informativos; el **mean pooling** resultó "*one of the most effective and stable aggregation strategies*"; y curiosamente el modelo afinado para dysarthria **no superó consistentemente** al base. Útil para justificar tu elección de pooling. (ScienceDirect + 2)
- **Klempíř et al. (2024), medRxiv 2024.04.10.24305599** — wav2vec 1.0 en clasificación cross-database y regresión (edad, caracteres/segundo) de PD, mostrando que las representaciones wav2vec superan a MFCC y generalizan entre idiomas. (PubMed Central) (medRxiv)

2.4 Deterioro cognitivo leve (MCI) y demencia multiclase

- **PROCESS Signal Processing Grand Challenge (ICASSP 2025)**, iniciado por University of Sheffield y University of Edinburgh — primer reto que añade **MCI** como clase (clasificación 3-clases: healthy/MCI/dementia + regresión de MMSE). Corpus de 82 controles sanos, 59 MCI y 16 casos de demencia (arXiv 2412.09928). El equipo ganador

integró rasgos interpretables + temporales de modelos preentrenados, logrando **F1 = 0.649** en clasificación y **RMSE = 2.628** en MMSE. Otros equipos combinaron eGeMAPS + HuBERT + GPT-4o (Macro-F1 0.751 superando baselines de WavLM, HuBERT y Wav2Vec 2.0). [\(IEEEICASSP + 5\)](#)

- **Zafar, Zhang, Shahin & Ahmed (ICASSP 2025)** — "Multi-Class Dementia Detection Using Acoustic Features": ensemble que alcanza Macro-F1 0.96 en dev y 0.99 en 5-fold CV pero **solo 0.64 en test** — otra ilustración del gap dev→test en datos clínicos pequeños. [\(IEEE Xplore\)](#)
- **TAUKADIAL (Interspeech 2024)**: MCI vs. NC en inglés/mandarín, con HuBERT y wav2vec2 como baselines. [\(arXiv\)](#)

2.5 ALS, dysarthria y habla patológica (puente hacia neurodegenerativas motoras)

La dysarthria es transversal a PD, ALS y parálisis cerebral, y es el dominio donde más se ha explorado PEFT sobre SSL:

- **Detección/severidad de dysarthria con wav2vec2** (arXiv 2309.14107, UA-Speech): mejores resultados con **embeddings de la primera capa**; mejora absoluta de 1.23 % sobre el espectrograma. [\(arXiv\)](#)
- **Clasificación de severidad speaker-independent** (arXiv 2403.00854): wav2vec2 + contrastive + multitask supera métodos tradicionales. [\(arxiv\)](#)
- **Structured Speaker-Deficiency Adaptation** (arXiv 2412.18832): **bloques adapter residuales bottleneck (RAB)** sobre HuBERT-large (UASpeech) y wav2vec2-conformer (DementiaBank), **dimensión de cuello de botella 256** (UASpeech) y 128–256 (DementiaBank). Es PEFT sobre SSL en datos clínicos, aunque para **ASR**, no clasificación de enfermedad. [\(arxiv\)](#)
- **ASR para Parkinson** (arXiv 2409.19818, Speech Accessibility Project): fine-tuning de wav2vec 2.0 reduce WER; el multitask con severidad como salida auxiliar ayuda en casos severos. [\(arxiv\)](#)

2.6 PEFT específicamente para clasificación neurodegenerativa: la laguna que tu tesis puede llenar

Este es el hallazgo más importante para tu posicionamiento. Tras búsqueda dedicada, la intersección {PEFT} × {SSL de habla} × {clasificación de enfermedad neurodegenerativa} es casi un vacío:

- **HASS** (Li et al., 2026, arXiv 2603.26795, UC Berkeley/UCSF) — detección de **afasia progresiva primaria logopénica (lvPPA)**, trastorno neurodegenerativo. Aplica **LoRA sobre wav2vec 2.0 base, exclusivamente en las proyecciones query y value** de la auto-atención. Datos: corpus clínicos (Baycrest, Hopkins PPA de DementiaBank) + corpus sintético. AUC 0.892 ± 0.076 , F1 0.800 ± 0.072 . **No reporta rango LoRA, ni % de parámetros, ni comparación PEFT vs. full FT** (su contribución es el dato sintético). Es el match más limpio con tus criterios de modelo.

- **VoxCog** (Feng, Xu, Lee, Narayanan, 2026, arXiv 2601.07999, USC) — clasificación de **AD y MCI** multilingüe (ADReSS, ADReSSo, NCMMSC, TAUkADIAL, etc.) con **LoRA + capas conv 1D** sobre backbones de fundación. Caveat: usa **Whisper-Large y MMS**, no wav2vec2/HuBERT/WavLM. Acc 87.5 %/F1 0.875 en ADReSS 2020; 85.92 %/0.859 en ADReSSo. Tampoco reporta rango ni comparación PEFT vs. full FT.

Plantilla metodológica de referencia: como ninguno de los dos detalla el rango/parámetros, el precedente cuantitativo más sólido de "LoRA sobre wav2vec2/XLSR" proviene de detección de audio falso (Wang et al., 2023, IJCAI-W / arXiv 2306.05617): **rango $r=4$ óptimo**, LoRA en Wq/Wv (o Wq/Wk/Wv), **reducción ~198× de parámetros entrenables** con caída de desempeño marginal. Y los benchmarks de PEFT en habla (PEFT-SER arXiv 2306.05350; ELP-Adapters arXiv 2407.21066, que con WavLM iguala o supera al full FT con **90 % menos parámetros**) te dan el respaldo empírico de que la idea funciona en clasificación de habla. (arxiv)

Recommendations

Etapla 0 — Reproduce el pipeline base (1-2 semanas). En tu PC, monta

`Wav2Vec2ForSequenceClassification` con `facebook/wav2vec2-base`, `freeze_feature_encoder()`, mean pooling y cabeza binaria. Verifica el flujo con un subconjunto pequeño (p. ej. ADReSS o PC-GITA si consigues acceso vía Orozco-Arroyave en U. de Antioquia, que te queda cerca). Si la GPU es limitada, empieza con **embeddings congelados + SVM/regresión logística** (rápido, pocos datos, baseline robusto que la literatura de PD usa con éxito).

Etapla 1 — Establece baselines de fine-tuning. Compara, con la **misma partición por hablante y 5-fold CV**: (a) embeddings congelados + clasificador clásico; (b) fine-tuning parcial (últimas 3 capas + cabeza, fp16); (c) full fine-tuning. Reporta accuracy, F1-macro y AUC. *Benchmark que cambia la decisión:* si (b) iguala a (c) sin diferencia significativa (como en Sampath et al. 2025), descarta el full FT por coste.

Etapla 2 — Introduce los tres métodos PEFT como tu contribución central. Implementa y compara **LoRA** (empieza $r \in \{4, 8, 16\}$ sobre q_proj/v_proj), **bottleneck adapters** (Pfeiffer y Houlsby, `reduction_factor` 16) y **prefix-tuning** (`num_virtual_tokens` $\in \{8, 16\}$). Mide para cada uno: F1/AUC vs. **% de parámetros entrenables** (esta curva eficiencia-desempeño es la figura estrella de tu tesis). *Benchmark:* el objetivo realista es igualar al full FT usando **<3 %** de parámetros (precedente en SER).

Etapla 3 — Valida contra el efecto Clever Hans. Haz al menos una evaluación **cross-corpus** (entrenar en un corpus, testear en otro) o cross-lingüística. Si tu accuracy intra-corpus es ~97 % pero cae a ~60-77 % cross-corpus, repórtalo explícitamente: es un resultado honesto y científicamente valioso, no un fracaso.

Etapla 4 — Posiciona tu originalidad. Dado que PEFT para clasificación neurodegenerativa con wav2vec2 está casi inexplorado (solo HASS y VoxCog en 2026, ninguno con comparación sistemática PEFT vs. full FT ni reporte de rango/parámetros), **una comparación rigurosa y**

reproducibile de adapters vs. LoRA vs. prefix-tuning en régimen de pocos datos clínicos es una contribución publicable (apunta a Interspeech o ICASSP SPGC). Reporta lo que ellos omitieron: rango, % de parámetros, y la comparación frente al full FT.

Decisiones por defecto recomendadas: modelo `wav2vec2-base` (no large, por datos limitados y tu hardware); congelar CNN; **mean pooling** (estable y simple, respaldado por la literatura PD); LR 1e-4 para PEFT; batch 1 + grad-accum 4; 15 épocas con warmup 0.1; F1-macro como métrica de selección. Considera además extraer la **primera capa Transformer** como representación alternativa, dado que múltiples estudios la señalan como la más informativa en habla patológica.

Caveats

- **Hardware y modelo:** full fine-tuning de wav2vec2-base puede dar OOM en GPUs de consumo (GTX/RTX 1080/2080 Ti) con batch $\geq$ 32; por eso PEFT y el fine-tuning parcial con fp16/caching son tu mejor opción en un PC personal (Sampath et al. 2025 lo documentan explícitamente).
- **Soporte de prefix-tuning para wav2vec2 es el más frágil** de los tres en `peft` (orientado a LLMs); si encuentras errores, usa la implementación de la librería `adapters` o prioriza LoRA y bottleneck adapters. No hallé ningún trabajo previo que use prefix-tuning para clasificación neurodegenerativa, así que es territorio experimental.
- **Riesgo de sobreajuste y fuga de datos:** con corpus de decenas de hablantes, la métrica intra-corpus engaña. Particiona **siempre por hablante** (nunca por grabación) para evitar que el modelo memorice voces. El gap dev $\rightarrow$ test (0.96 $\rightarrow$ 0.64 en Zafar et al. 2025) es la norma, no la excepción.
- **Cifras de la literatura no son directamente comparables:** difieren en corpus, idioma, tarea (binaria vs. 3-clases), y en si usan embeddings congelados, full FT o features lingüísticos. No mezcles un 97 % del Pitt corpus con un 64 % del PROCESS test como si midieran lo mismo.
- **Los dos preprints PEFT-neurodegenerativos (HASS, VoxCog) son de 2026 y no estaban revisados por pares al momento de esta búsqueda;** cítalos como preprints de arXiv y verifica si hay versión publicada antes de la defensa.
- **Acceso a datos:** ADReSS/ADReSSo/Pitt requieren membresía DementiaBank/TalkBank; PC-GITA es bajo solicitud al Prof. Juan Rafael Orozco-Arroyave (U. de Antioquia). Gestiona estos accesos con antelación. `medRxiv`

Citas bibliográficas completas (para tu tesis)

- Baevski, A., Zhou, Y., Mohamed, A., & Auli, M. (2020). *wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations*. NeurIPS 33, 12449–12460. arXiv:2006.11477.
- Houlisby, N., Giurgiu, A., Jastrzebski, S., Morrone, B., de Laroussilhe, Q., Gesmundo, A., Attariyan, M., & Gelly, S. (2019). *Parameter-Efficient Transfer Learning for NLP*. ICML.

- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv:2106.09685 (ICLR 2022).
- Li, X. L., & Liang, P. (2021). *Prefix-Tuning: Optimizing Continuous Prompts for Generation*. ACL. arXiv:2101.00190.
- Lester, B., Al-Rfou, R., & Constant, N. (2021). *The Power of Scale for Parameter-Efficient Prompt Tuning*. EMNLP.
- Pfeiffer, J., et al. (2020/2021). *AdapterHub / AdapterFusion*. EMNLP/EACL.
- Poth, C., Sterz, H., Paul, I., et al. (2023). *Adapters: A Unified Library for Parameter-Efficient and Modular Transfer Learning*. EMNLP (system demo). arXiv:2311.11077.
- Sampath, A., Tavernor, J., & Mower Provost, E. (2025). *Efficient Finetuning for Dimensional Speech Emotion Recognition in the Age of Transformers*. arXiv:2503.03756 (IEEE).
- Feng, T., et al. (2024). *PEFT-SER: On the Use of Parameter Efficient Transfer Learning Approaches for Speech Emotion Recognition Using Pre-trained Speech Models*. arXiv:2306.05350 (ACII).
- Inoue, N., Otake, S., Hirose, T., Ohi, M., & Kawakami, R. (2024). *ELP-Adapters: Parameter Efficient Adapter Tuning for Various Speech Processing Tasks*. arXiv:2407.21066.
- Wang, C., Yi, J., Zhang, X., Tao, J., Xu, L., & Fu, R. (2023). *Low-rank Adaptation Method for Wav2vec2-based Fake Audio Detection*. IJCAI 2023 Workshop (DADA). arXiv:2306.05617.
- Agbavor, F., & Liang, H. (2022). *Predicting dementia from spontaneous speech using large language models*. PLOS Digital Health, 1(12):e0000168.
- Zhu, Y., Obyat, A., Liang, X., Batsis, J. A., & Roth, R. M. (2021). *WavBERT: Exploiting Semantic and Non-semantic Speech using Wav2vec and BERT for Dementia Detection*. Interspeech 2021, 3790–3794. doi:10.21437/interspeech.2021-332.
- Liu, Y., Feng, et al. (2024). *Clever Hans Effect Found in Automatic Detection of Alzheimer's Disease through Speech*. arXiv:2406.07410.
- Gallo-Aristizábal, J. D., Escobar-Grisales, D., Ríos-Urrego, C. D., Nöth, E., & Orozco-Arroyave, J. R. (2024). *Automatic Classification of Parkinson's Disease Using Wav2vec Embeddings at Phoneme, Syllable, and Word Levels*. TSD 2024, LNCS 15049. doi:10.1007/978-3-031-70566-3_27.
- Klempíř, O., Skryjová, A., Tichopad, A., & Krupička, R. (2025). *Ranking Pretrained Speech Embeddings in Parkinson's Disease Detection*. medRxiv 2025.01.29.25321319.
- Luz, S., Haider, F., Fromm, D., Lazarou, I., Kompatsiaris, I., & MacWhinney, B. (2024). *An Overview of the ADReSS-M Signal Processing Grand Challenge*. IEEE Open Journal of Signal Processing. doi:10.1109/OJSP.2024.3378595.
- Tao, F., Mirheidari, B., Pahar, M., et al. (2025). *Early Dementia Detection Using Multiple Spontaneous Speech Prompts: The PROCESS Challenge*. ICASSP 2025.
- Li, H., et al. (2026). *HASS: Hierarchical Simulation of Logopenic Aphasic Speech for Scalable PPA Detection*. arXiv:2603.26795 (preprint).
- Feng, T., Xu, A., Lee, J., & Narayanan, S. (2026). *VoxCog: Towards End-to-End Multilingual Cognitive Impairment Classification through Dialectal Knowledge*. arXiv:2601.07999

(preprint).